VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Microcontroller

Lab Report

Lab 2: Timer Interrupt and LED Scanning

Student(s): Âu Lê Quang Trí - 2453300

HO CHI MINH CITY, OCTOBER 2026

Contents

1 Introduction and Timer Interrupt Setup **3**
1.1 Principles of LED Scanning (Multiplexing) 3
1.2 Timer 2 Configuration 3

2 Exercise 1: Interfacing Two 7-Segment LEDs **5**

3 Exercise 2: Four 7-Segment LEDs with Blinking DOT **8**

4 Exercise 3: Implementation of Function `update7SEG` **11**

5 Exercise 4: Setting the Scanning Frequency to 1 Hz **13**

6 Exercise 5: Digital Clock with Hour and Minute Buffer **15**

7 Exercise 6: Software Timer Implementation and Analysis **17**
7.1 Software Timer Concept 17

8 Exercise 7: Upgrading Digital Clock with Software Timer **20**

9 Exercise 8: Moving Display Scanning to `main()` **22**

10 Exercise 9: 8x8 LED Matrix Display **25**

11 Exercise 10: LED Matrix Animation (Horizontal Scrolling) **28**

12 Source Code and Repository **30**

1 Introduction and Timer Interrupt Setup

1.1 Principles of LED Scanning (Multiplexing)

In embedded systems, driving multiple 7-segment displays or LED matrices using individual GPIO pins quickly exhausts the microcontroller's I/O pins. The technique of **Time-Division Multiplexing (LED scanning)** overcomes this limitation:

- All identical segment cathodes (A, B, C, D, E, F, G) from each display are tied together to a common 7-bit bus (PB0–PB6).
- The common anode terminal of each display is connected to an independent control switch (such as a PNP transistor or ULN2803 Darlington array), driven by MCU control pins (PA6–PA9).
- At any given instant t, only one display is powered on with its corresponding segment pattern. By cycling through all displays at a sufficiently high frequency ($f > 30\,\text{Hz}$), the human eye perceives continuous illumination without flicker due to **Persistence of Vision (POV)**.

1.2 Timer 2 Configuration

Hardware Timer 2 (TIM2) of the STM32F103C6 is configured to generate periodic interrupts every 10 ms:

- **Clock Source:** Internal High-Speed Clock (HSI) running at $f_{SYS} = 8\,\text{MHz}$.
- **Prescaler (PSC):** Set to 7999. The timer counting frequency is:

$$f_{TIM} = \frac{f_{SYS}}{\text{PSC} + 1} = \frac{8\,\text{MHz}}{7999 + 1} = 1000\,\text{Hz} \quad (T_{tick} = 1\,\text{ms}) \tag{1.1}$$

- **Auto-Reload Register / Period (ARR):** Set to 9. The counter counts from 0 up to 9 (10 counts per overflow event):

$$f_{IT} = \frac{f_{TIM}}{\text{ARR} + 1} = \frac{1000\,\text{Hz}}{9 + 1} = 100\,\text{Hz} \tag{1.2}$$

$$T_{IT} = \frac{1}{f_{IT}} = \frac{1}{100\,\text{Hz}} = 10\,\text{ms} \tag{1.3}$$

- **NVIC Settings:** TIM2 global interrupt is enabled in the NVIC settings.



- **Startup:** Initiated in `main()` via `HAL_TIM_Base_Start_IT(&htim2)`.

2 Exercise 1: Interfacing Two 7-Segment LEDs

Report 1: Schematic

The Proteus schematic implementing two common-anode 7-segment displays controlled via PNP transistors (PA6 and PA7) with segment lines connected to PB0–PB6 is illustrated below:

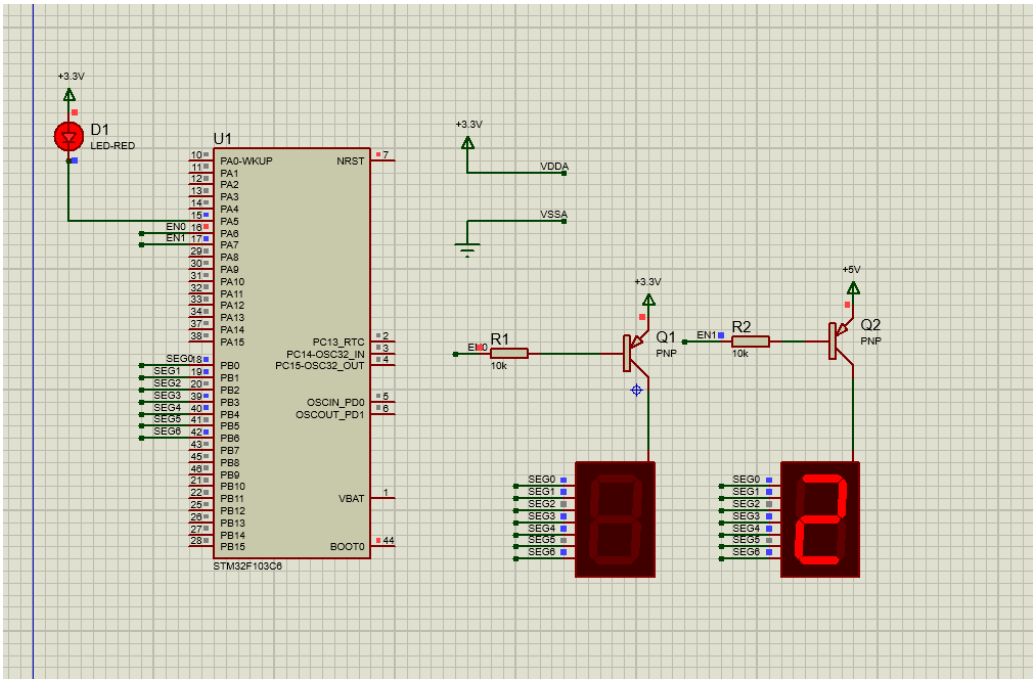


Figure 2.1: Proteus simulation schematic for Exercise 1 (Two 7-segment displays)

Report 2: Source Code

The segment mapping table and the 7-segment decoder function are defined as follows (active-low segments):

```
const uint8_t seg_code[10] = {
    0x40, // 0: 0b01000000
    0x79, // 1: 0b01111001
    0x24, // 2: 0b00100100
    0x30, // 3: 0b00110000
    0x19, // 4: 0b00011001
    0x12, // 5: 0b00010010
```

```
    0x02, // 6: 0b00000010
    0x78, // 7: 0b01111000
    0x00, // 8: 0b00000000
    0x10  // 9: 0b00010000
};

void display7SEG(int num) {
    if (num < 0 || num > 9) return;
    GPIOB->ODR = seg_code[num];
}
```

The interrupt service routine `HAL_TIM_PeriodElapsedCallback` toggles between the two displays every 500 ms (50×10 ms):

```
int counter = 1;   // Initialized to 1 so the first digit displays
immediately
int status  = 0;

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    counter--;
    if (counter <= 0) {
        counter = 50;  // 50 x 10ms = 500ms

        // Turn both displays off first to prevent ghosting
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);

        if (status == 0) {
            display7SEG(1);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);  //
            Enable 1st display
            status = 1;
        } else {
            display7SEG(2);
```

```
20              HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);  // Enable 2nd display
21              status = 0;
22          }
23      }
24  }
```

Short Question: What is the frequency of the scanning process?

Answer: Each 7-segment display is activated for an interval of $T_{\text{display}} = 500\,\text{ms} = 0.5\,\text{s}$. Because there are $N = 2$ displays in this multiplexed system, a complete cycle scanning both displays requires:

$$T_{\text{scan}} = N \times T_{\text{display}} = 2 \times 0.5\,\text{s} = 1.0\,\text{s} \tag{2.1}$$

Therefore, the full cycle scanning frequency is:

$$f_{\text{scan}} = \frac{1}{T_{\text{scan}}} = \frac{1}{1.0\,\text{s}} = 1.0\,\text{Hz} \tag{2.2}$$

(The digit-switching transition rate between successive LEDs is 2 Hz).

3 Exercise 2: Four 7-Segment LEDs with Blinking DOT

Report 1: Schematic

The Proteus schematic is extended to four 7-segment displays (enabled by PA6, PA7, PA8, PA9) and two colon DOT LEDs connected to PA4:

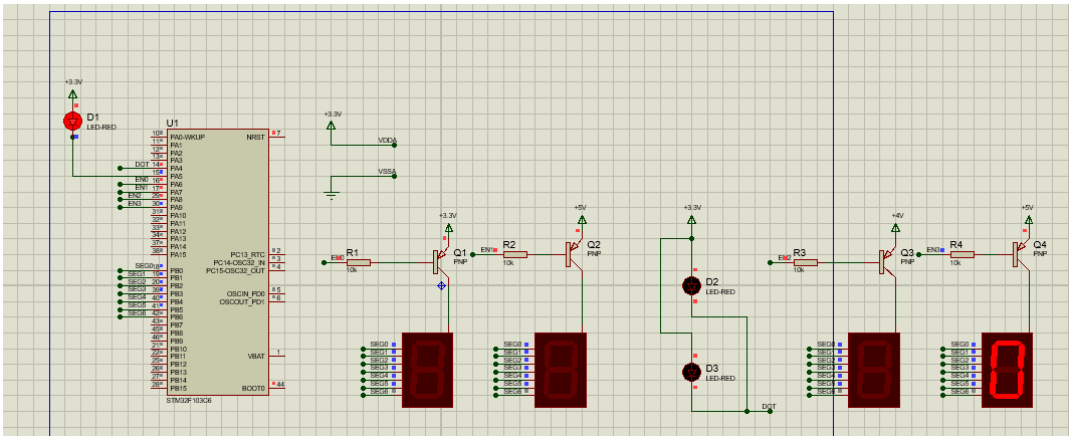


Figure 3.1: Proteus schematic for Exercise 2 (Four 7-segment displays and DOT LEDs)

Report 2: Source Code

The displays show "1", "2", "3", "0" (representing 12:30), switching every 500 ms. The DOT LEDs toggle every second ($100 \times 10\,\text{ms} = 1000\,\text{ms}$):

```
int counter = 1;
int status  = 0;
int dot_counter = 100;

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    // Handle DOT LED blinking every 1 second
    dot_counter--;
    if (dot_counter <= 0) {
        dot_counter = 100;
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
    }
```

```
13
14      // Handle 7-segment scanning every 500ms
15      counter--;
16      if (counter <= 0) {
17          counter = 50;  // 50 x 10ms = 500ms
18
19          // Turn all four displays off first to eliminate ghosting
20          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
21          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);
22          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
23          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET);
24
25          switch (status) {
26              case 0:
27                  display7SEG(1);
28                  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
29                  break;
30              case 1:
31                  display7SEG(2);
32                  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);
33                  break;
34              case 2:
35                  display7SEG(3);
36                  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);
37                  break;
38              case 3:
39                  display7SEG(0);
40                  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET);
41                  break;
42          }
43          status = (status + 1) % 4;
44      }
45  }
```

Short Question: What is the frequency of the scanning process?

Answer: Each of the $N = 4$ displays stays active for $T_{\text{display}} = 500\,\text{ms} = 0.5\,\text{s}$. The period required to complete one full scanning round of all 4 displays is:

$$T_{\text{scan}} = N \times T_{\text{display}} = 4 \times 0.5\,\text{s} = 2.0\,\text{s} \tag{3.1}$$

Consequently, the full-cycle scanning frequency is:

$$f_{\text{scan}} = \frac{1}{T_{\text{scan}}} = \frac{1}{2.0\,\text{s}} = 0.5\,\text{Hz} \tag{3.2}$$

4 Exercise 3: Implementation of Function update7SEG

Report 1: Source Code of update7SEG

To decouple the digit-rendering logic from the timing loop, the buffer `led_buffer` and the modular function `update7SEG(int index)` are implemented:

```
const int MAX_LED = 4;
int index_led = 0;
int led_buffer[4] = {1, 2, 3, 4};

void update7SEG(int index) {
    // Turn off all displays to prevent ghosting artifacts
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET);

    switch (index) {
        case 0:
            display7SEG(led_buffer[0]);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
            break;
        case 1:
            display7SEG(led_buffer[1]);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);
            break;
        case 2:
            display7SEG(led_buffer[2]);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);
            break;
        case 3:
            display7SEG(led_buffer[3]);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET);
            break;
        default:
```



```
            break;
        }
    }
```

Report 2: Source Code in HAL_TIM_PeriodElapsedCallback

The callback invokes update7SEG and increments index_led cyclically:

```
int counter = 1;
int dot_counter = 100;

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    dot_counter--;
    if (dot_counter <= 0) {
        dot_counter = 100;
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
    }

    counter--;
    if (counter <= 0) {
        counter = 50;  // 50 x 10ms = 500ms
        update7SEG(index_led++);
        if (index_led >= MAX_LED) index_led = 0;
    }
}
```

5 Exercise 4: Setting the Scanning Frequency to 1 Hz

Report 1: Source Code in HAL_TIM_PeriodElapsedCallback

To achieve an overall scanning frequency of $f_{\text{scan}} = 1\,\text{Hz}$ across the 4 displays:

$$T_{\text{scan}} = \frac{1}{f_{\text{scan}}} = 1.0\,\text{s} = 1000\,\text{ms} \tag{5.1}$$

The time allocation for each individual display must therefore be:

$$T_{\text{per_LED}} = \frac{T_{\text{scan}}}{4} = \frac{1000\,\text{ms}}{4} = 250\,\text{ms} \tag{5.2}$$

With the hardware timer tick period of 10 ms, the software counter must be reloaded with:

$$\text{counter} = \frac{250\,\text{ms}}{10\,\text{ms}} = 25 \tag{5.3}$$

The implemented interrupt service routine is:

```
int counter = 1;
int dot_counter = 100;

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    // DOT LED continues toggling every second (100 x 10ms = 1000ms)
    dot_counter--;
    if (dot_counter <= 0) {
        dot_counter = 100;
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
    }

    // 4-digit display scanning at 1Hz (250ms per digit)
    counter--;
    if (counter <= 0) {
        counter = 25;  // 25 x 10ms = 250ms -> 4 x 250ms = 1000ms =
        1s (1Hz)
```



```
17          update7SEG(index_led++);
18          if (index_led >= MAX_LED) index_led = 0;
19      }
20  }
```

6 Exercise 5: Digital Clock with Hour and Minute Buffer

Report 1: Source Code in `updateClockBuffer`

To implement a digital clock, the time variables `hour`, `minute`, and `second` are maintained. The function `updateClockBuffer` extracts the tens and units digits of the current time and populates the 4-element array `led_buffer`:

```
int hour = 15, minute = 8, second = 50;

void updateClockBuffer(void) {
    led_buffer[0] = hour / 10;    // Tens of the hour
    led_buffer[1] = hour % 10;    // Units of the hour
    led_buffer[2] = minute / 10;  // Tens of the minute
    led_buffer[3] = minute % 10;  // Units of the minute
}
```

In the `main()` loop, the clock advances every second using `HAL_Delay(1000)`:

```
while (1)
{
    second++;
    if (second >= 60) {
        second = 0;
        minute++;
    }
    if (minute >= 60) {
        minute = 0;
        hour++;
    }
    if (hour >= 24) {
        hour = 0;
    }
    updateClockBuffer();
    HAL_Delay(1000);
```

```
17 }
```

7 Exercise 6: Software Timer Implementation and Analysis

7.1 Software Timer Concept

Relying on blocking functions like `HAL_Delay()` stalls the processor and wastes CPU cycles. Furthermore, placing excessive computation inside the interrupt handler can cause interrupt latency and nested interrupt crashes. A **software timer** maintains an integer counter decremented in the 10 ms hardware timer ISR. When the counter reaches zero, a software flag is raised, signaling the background `main()` loop to execute the scheduled task non-blockingly:

```
#define TIMER_CYCLE 10  // ms per timer interrupt tick

int timer0_counter = 0;
volatile int timer0_flag = 0;

void setTimer0(int duration) {
    timer0_counter = duration / TIMER_CYCLE;
    timer0_flag = 0;
}

void timer_run(void) {
    if (timer0_counter > 0) {
        timer0_counter--;
        if (timer0_counter == 0) timer0_flag = 1;
    }
}
```

The test code in `main()`:

```
setTimer0(1000);
while (1) {
    if (timer0_flag == 1) {
        HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
        setTimer0(2000);
```

```
6       }
7   }
```

Report 1: Analysis if line 1 (`setTimer0(1000)`) is missed

Question: If in line 1 of the code above is missed, what happens after that and why?
Answer:

- **Observation:** The LED will never toggle and will remain permanently in its initial state.
- **Cause:** `timer0_counter` is initialized to 0, and `timer0_flag` is initialized to 0. When hardware interrupts occur every 10 ms, the ISR invokes `timer_run()`. Inside `timer_run()`, the condition `if (timer0_counter > 0)` evaluates to **false** ($0 > 0$ is false). Therefore, `timer0_counter` is never decremented, and the assignment `timer0_flag = 1` is never reached. Consequently, in `main()`, the polling condition `if (timer0_flag == 1)` will never become true, stalling the toggle mechanism completely.

Report 2: Analysis if line 1 is changed to `setTimer0(1)`

Question: If in line 1 of the code above is changed to `setTimer0(1)`, what happens after that and why?
Answer:

- **Observation:** The LED will also never toggle and remains stuck in its initial state.
- **Cause:** In C programming, integer division truncates fractions toward zero:

$$\text{timer0_counter} = \frac{\text{duration}}{\text{TIMER_CYCLE}} = \frac{1}{10} = 0 \tag{7.1}$$

  Because `timer0_counter` evaluates to 0 and `timer0_flag` is cleared to 0, the subsequent interrupt executions in `timer_run()` encounter timer0_counter = 0. Since 0 is not strictly greater than 0, the counter is never decremented and `timer0_flag` is never set to 1. The condition in the `while(1)` loop is never triggered.

Report 3: Analysis if line 1 is changed to `setTimer0(10)`

Question: If in line 1 of the code above is changed to `setTimer0(10)`, what is changed compared to the first two questions and why?



Answer:

- **Observation:** The LED successfully toggles! Specifically, the first toggle occurs almost immediately (exactly 10 ms after startup), after which the LED toggles periodically every 2000 ms (2 seconds).
- **Cause:** With duration = 10 ms, the integer division yields:

$$\text{timer0_counter} = \frac{10}{10} = 1 \tag{7.2}$$

  At the very first hardware timer interrupt (after 10 ms), `timer_run()` decrements `timer0_counter` from 1 to 0. Since it reaches 0, `timer0_flag` is set to 1. In the `main()` loop, `if (timer0_flag == 1)` is satisfied: the LED toggles at $t = 10$ ms, and the software timer is immediately rearmed with `setTimer0(2000)` (`timer0_counter` $= 2000/10 = 200$). From then on, the LED blinks reliably with a half-period of 2 s.

8 Exercise 7: Upgrading Digital Clock with Software Timer

Report 1: Source Code in the `while (1)` Loop

The blocking delay `HAL_Delay(1000)` is replaced by calling `setTimer0(1000)` and checking `timer0_flag`. In addition, the DOT LED toggle is moved to `main()`:

```
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start_IT(&htim2);
updateClockBuffer();
setTimer0(1000);
/* USER CODE END 2 */

while (1)
{
    /* USER CODE BEGIN 3 */
    if (timer0_flag == 1) {
        setTimer0(1000);  // Rearm software timer for the next 1
        second interval
        second++;
        if (second >= 60) {
            second = 0;
            minute++;
        }
        if (minute >= 60) {
            minute = 0;
            hour++;
        }
        if (hour >= 24) {
            hour = 0;
        }
        updateClockBuffer();
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);  // Toggle colon DOT in
        main()
    }
```



```
}
```

The timer ISR handles decrementing the timer counter and scanning the displays:

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    timer_run();
    counter--;
    if (counter <= 0) {
        counter = 25;  // 250ms scanning rate per digit
        update7SEG(index_led++);
        if (index_led >= MAX_LED) index_led = 0;
    }
}
```

9 Exercise 8: Moving Display Scanning to `main()`

Report 1: Source Code in `main()` and Software Timer System

To optimize interrupt performance, all display updating is moved out of the ISR into `main()`. The timer interrupt is dedicated solely to ticking software timers. An array-based multi-timer architecture is implemented:

```
#define TIMER_CYCLE  10  // 10ms tick
#define NO_OF_TIMERS 4

#define TIMER_SECOND 0   // Clock tick & DOT (1000ms)
#define TIMER_SCAN7  1   // 7-segment multiplexing (250ms)
#define TIMER_MATRIX 2   // LED Matrix scanning
#define TIMER_SHIFT  3   // Matrix animation

int timer_counter[NO_OF_TIMERS];
volatile int timer_flag[NO_OF_TIMERS];

void setTimer(int index, int duration) {
    timer_counter[index] = duration / TIMER_CYCLE;
    timer_flag[index] = 0;
}

void timer_run(void) {
    for (int i = 0; i < NO_OF_TIMERS; i++) {
        if (timer_counter[i] > 0) {
            timer_counter[i]--;
            if (timer_counter[i] == 0) timer_flag[i] = 1;
        }
    }
}
```

The ISR is now extremely lean ($O(1)$ operations with minimal register usage):

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    timer_run();
}
```

The complete non-blocking background loop in `main()`:

```
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start_IT(&htim2);
updateClockBuffer();
setTimer(TIMER_SECOND, 1000);
setTimer(TIMER_SCAN7, 250);
/* USER CODE END 2 */

while (1)
{
    // Task 1: 1-second clock increment and DOT blinking
    if (timer_flag[TIMER_SECOND] == 1) {
        setTimer(TIMER_SECOND, 1000);
        second++;
        if (second >= 60) {
            second = 0;
            minute++;
        }
        if (minute >= 60) {
            minute = 0;
            hour++;
        }
        if (hour >= 24) {
            hour = 0;
        }
        updateClockBuffer();
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
    }

```



```
29      // Task 2: 7-segment display scanning every 250ms
30      if (timer_flag[TIMER_SCAN7] == 1) {
31          setTimer(TIMER_SCAN7, 250);
32          update7SEG(index_led++);
33          if (index_led >= MAX_LED) index_led = 0;
34      }
35  }
```

10 Exercise 9: 8x8 LED Matrix Display

Report 1: Schematic

The Proteus schematic incorporates the `MATRIX-8X8-RED` and a `ULN2803` Darlington transistor array. The row pins (ROW0–ROW7) are wired to PB8–PB15, while the column enable lines (ENM0–ENM7) are wired to GPIOA pins:

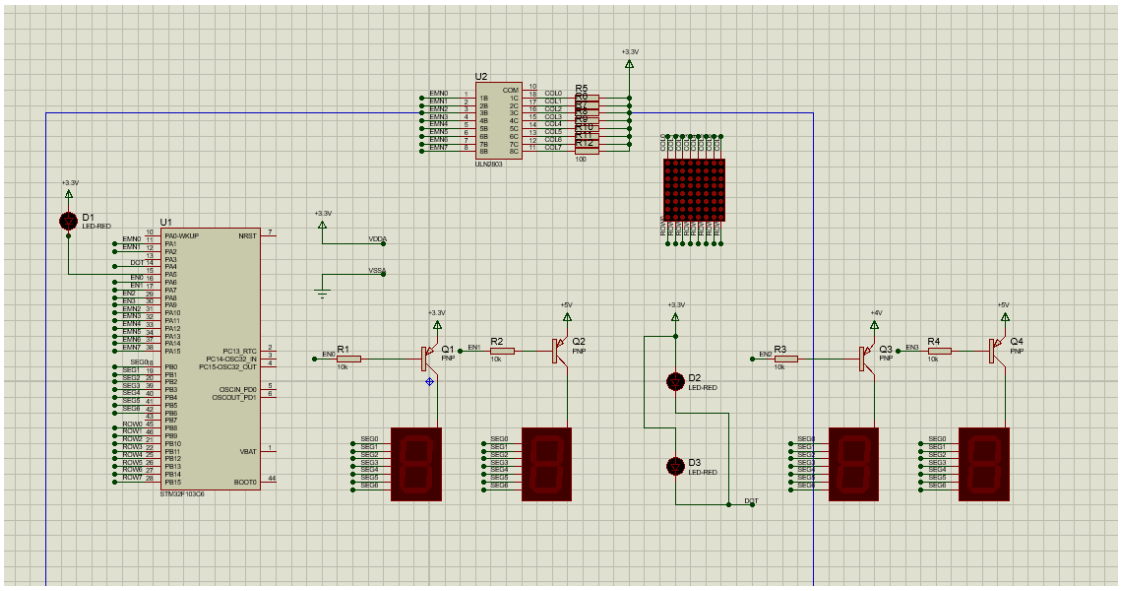


Figure 10.1: Proteus schematic for Exercise 9 (8x8 LED Matrix with ULN2803 driver)

Report 2: Source Code of `updateLEDMatrix`

To display the character "A", the bitmap buffer is defined per column (with bit 0 corresponding to ROW0 at the top). Because the 7-segment displays use PB0–PB6, the 7-segment display function is refined to modify only bits 0–6 of `GPIOB->ODR`, preserving the matrix row data on PB8–PB15:

```
1  void display7SEG(int num) {
2      if (num < 0 || num > 9) return;
3      GPIOB->ODR = (GPIOB->ODR & ~0x007F) | seg_code[num];
4  }
5
6  const int MAX_LED_MATRIX = 8;
7  int index_led_matrix = 0;
```

```
8
9  // Column bitmap for character 'A' (columns 0 to 7)
10 uint8_t matrix_buffer[8] = {0xFC, 0xFE, 0x13, 0x11, 0x11, 0x13, 0xFE,
   0xFC};
11
12 const uint16_t enm_pin[8] = {
13     GPIO_PIN_1,  GPIO_PIN_2,  GPIO_PIN_10, GPIO_PIN_11,
14     GPIO_PIN_12, GPIO_PIN_13, GPIO_PIN_14, GPIO_PIN_15
15 };
16
17 #define ENM_ON     GPIO_PIN_RESET  // ULN2803 inverts input: LOW on
   MCU enables driver
18 #define ENM_OFF    GPIO_PIN_SET
19 #define ROW_INVERT 1
20
21 static void matrixShow(int col) {
22     // 1. Turn off all columns to avoid ghosting
23     for (int i = 0; i < 8; i++) {
24         HAL_GPIO_WritePin(GPIOA, enm_pin[i], ENM_OFF);
25     }
26     // 2. Output row pattern onto PB8..PB15
27     uint8_t data = matrix_buffer[col];
28     if (ROW_INVERT) data = (uint8_t)~data;
29     GPIOB->ODR = (GPIOB->ODR & ~0xFF00) | ((uint32_t)data << 8);
30
31     // 3. Enable the active column
32     HAL_GPIO_WritePin(GPIOA, enm_pin[col], ENM_ON);
33 }
34
35 void updateLEDMatrix(int index) {
36     switch (index) {
37         case 0: matrixShow(0); break;
38         case 1: matrixShow(1); break;
39         case 2: matrixShow(2); break;
40         case 3: matrixShow(3); break;
```

```
41          case 4: matrixShow(4); break;
42          case 5: matrixShow(5); break;
43          case 6: matrixShow(6); break;
44          case 7: matrixShow(7); break;
45          default: break;
46      }
47  }
```

In `main()`, the matrix is scanned every 10 ms (`setTimer(TIMER_MATRIX, 10)`):

```
1  if (timer_flag[TIMER_MATRIX] == 1) {
2      setTimer(TIMER_MATRIX, 10);
3      updateLEDMatrix(index_led_matrix++);
4      if (index_led_matrix >= MAX_LED_MATRIX) index_led_matrix = 0;
5  }
```

11 Exercise 10: LED Matrix Animation (Horizontal Scrolling)

Report 1: Animation Solution Description and Source Code

Algorithm Description: To create a smooth left-scrolling animation of character "A":

1. A circular column shift buffer is implemented in function `shiftLeft()`.
2. Every column i ($0 \leq i < 7$) copies the bitmap data of its right neighbor $i + 1$:

$$\text{matrix_buffer}[i] = \text{matrix_buffer}[i + 1] \tag{11.1}$$

3. The leftmost column (`matrix_buffer[0]`), which shifts out of the visible screen, wraps around to the rightmost column (`matrix_buffer[7]`).
4. **Decoupled Timing:** The column multiplexing runs rapidly at 10 ms per column (yielding a full matrix refresh rate of $f_{\text{refresh}} = \frac{1}{8 \times 10\,\text{ms}} = 12.5\,\text{Hz}$), ensuring crisp POV illumination. The scrolling animation is driven by a independent software timer (`TIMER_SHIFT`) set to 200 ms per step, generating a fluid shift motion across the matrix.

The implementation of `shiftLeft()` and the main scheduling logic:

```
void shiftLeft(void) {
    uint8_t first = matrix_buffer[0];
    for (int i = 0; i < MAX_LED_MATRIX - 1; i++) {
        matrix_buffer[i] = matrix_buffer[i + 1];  // Shift left
    }
    matrix_buffer[MAX_LED_MATRIX - 1] = first;    // Wrap around to
    rightmost column
}
```

The complete scheduling loop in `main()`:

```c
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start_IT(&htim2);
updateClockBuffer();
setTimer(TIMER_SECOND, 1000);
setTimer(TIMER_SCAN7, 250);
setTimer(TIMER_MATRIX, 10);
setTimer(TIMER_SHIFT, 200);
/* USER CODE END 2 */

while (1)
{
    // 1. Clock increment and DOT toggle (1000ms)
    if (timer_flag[TIMER_SECOND] == 1) {
        setTimer(TIMER_SECOND, 1000);
        second++;
        if (second >= 60) {
            second = 0;
            minute++;
        }
        if (minute >= 60) {
            minute = 0;
            hour++;
        }
        if (hour >= 24) {
            hour = 0;
        }
        updateClockBuffer();
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
    }

    // 2. 7-segment scanning (250ms)
    if (timer_flag[TIMER_SCAN7] == 1) {
        setTimer(TIMER_SCAN7, 250);
        update7SEG(index_led++);
        if (index_led >= MAX_LED) index_led = 0;
```

```
    }

    // 3. LED matrix column scanning (10ms)
    if (timer_flag[TIMER_MATRIX] == 1) {
        setTimer(TIMER_MATRIX, 10);
        updateLEDMatrix(index_led_matrix++);
        if (index_led_matrix >= MAX_LED_MATRIX) index_led_matrix = 0;
    }

    // 4. Matrix animation shift left (200ms)
    if (timer_flag[TIMER_SHIFT] == 1) {
        setTimer(TIMER_SHIFT, 200);
        shiftLeft();
    }
}
```

12 Source Code and Repository

All STM32CubeIDE project files, C source code, Proteus simulation circuits, and LaTeX report files are maintained at our GitHub repository:

https://github.com/aulequangtri/microprocessor_lab